

Team Phalanx AI: Introduction to Deep Learning Tools

1. Introduction

This lab showed us how image recognition systems work without writing code. We used Google Colab with an existing vision program named VGG16. We set up our pictures, ran the program, and changed the images to see when the computer fails. Our goal was simple: see what the program looks at, find its limits, and track its blind spots.

2. Overview of the Pre-Built Model

We used VGG16, an image recognition tool trained on 1.4 million labeled photos across 1,000 everyday categories. It contains 138 million internal connection settings.

Olis showed the step-by-step setup that turns a raw photo into a final list of guesses. The setup works in stages. Early stages look for basic marks like straight lines, borders, and textures. Later stages combine those marks into larger shapes like ears, wheels, or corners. The tool holds no real idea of what an object is. It looks for matching arrangements of light and dark spots.

3. Data Loading and Preprocessing

The program requires every image to follow exact dimensions. We converted every photo to 224 pixels tall by 224 pixels wide, matched the three color channels, and scaled the color values to match the original training files.

Feeding the program raw photos without these changes causes an error or creates bad output. Olis included a visual chart showing food items like pizza, fries, and salad. In raw form, the photos look like a scattered mess of color points. After passing through the network stages, the points pull apart into neat groups. Prepping the image allows the tool to sort the shapes into those groups.

4. Making Predictions

Each team member tested original photos to see what the tool guessed.

- **Robert B. Thompson:** I uploaded a photo on a dock holding a small fish on a line, plus a photo of a wolf. The dock photo returned "picket fence." The parallel lines on the wooden dock boards filled most of the photo, so the program focused on those lines instead of me or the fish. The wolf photo returned timber wolf (82.0%), red wolf (8.0%), and coyote (2.5%).
- **Njeh Ababio:** Uploaded a dog picture and formatted the size to 224x224. The assignment did not provide a sample photo. Finding an outside image proved that feeding the model the wrong size immediately stops the process.
- **Olis Bahari:** Tested a photo of a striped cat. The program returned tiger cat (49.7%), tabby cat (30.6%), and Egyptian cat (18.6%). When the program is unsure, its backup choices stay within the same animal family.

Each test proved the same point: the program matches shapes it saw before; it does not understand the scene.

5. Understanding Predictions: Sensitivity Analysis

I tested the wolf photo by rotating the frame and adding visual static to measure when the program fails.

Rotation Test

Rotation	Top Guess	Confidence
0°	timber wolf	82.0%
15°	timber wolf	87.4%
30°	timber wolf	53.0%
45°	timber wolf	24.3%
90°	bustard (bird)	23.8%

Confidence held through 15 degrees, then dropped at 30 degrees. At 90 degrees, the wolf vanished from the top choices. The program expects wolves to stand on their feet. Turning the photo sideways scrambles the outline.

Noise Test

Static Level	Top Guess	Confidence
0	timber wolf	82.0%
10	timber wolf	78.6%
25	timber wolf	71.9%
50	coyote	53.9%

The confidence dropped slowly, then switched to coyote at level 50 static. Wolves and coyotes look similar. When static blurred the fur details, the tool picked the next closest match.

These tests match the points Olis noted. Standard image tools break down when you tilt them, and static disrupts the fine details long before human eyes lose track of the subject.

6. Team Findings

- **The tool matches surface patterns, not meaning.** My dock photo returned "picket fence" because repetitive wood planks filled the frame. Olis saw the same pattern with the cat; the top three answers stayed grouped in the cat family.
- **Sizing and cleanup are mandatory.** As Njeh found, skipping the 224x224 size prevents the model from reading the picture. Olis showed that raw pictures are chaotic until the system cleans and groups them.
- **Tilting an image breaks the system faster than static.** A 30 degree tilt dropped confidence by 29 percentage points. Level 25 static dropped confidence by only 10.1 percentage points. The system relies on upright shapes more than clear pixels.

7. Feedback and Assessment

The setup let us test how computer vision behaves without writing computer code. Providing sample photos inside the lesson would save setup time, though preparing our own files made the size rules clear.

We confirmed the full pipeline: feed the picture, resize the dimensions, run the check, and locate where the model fails. These tests show where vision programs work and where they break.